swayam

sagarkadyan823@gmail.com ⌄

NPTEL » The Joy of Computing using Python

Announcements About the Course Q&A Progress Mentor Review Assignment Course Recommendations

If already registered, click to check your payment status

Course outline

- **About NPTEL**
- **How does an NPTEL online course work?**
- **Week 1**
- **Week 2**
- **Week 3**
  - Lists Part 1 : Introduction
  - Lists Part 2 : Manipulation
  - Lists Part 3 : Operations
  - Lists Part 4 : Slicing
  - Loops and Conditionals : Fizzbuzz 01
  - Loops and Conditionals : Fizzbuzz 02
  - Crowd Computing - Just estimate 01
  - Crowd Computing - Just estimate 02
  - Crowd Computing - Just estimate 03
  - Crowd Computing - Just estimate 04
  - Crowd Computing - Just estimate 05
  - Crowd Computing - Just estimate 06
  - Permutations - Jumbled Words 01
  - Permutations - Jumbled Words 02
  - Permutations - Jumbled Words 03
  - Theory of Evolution 01
  - Theory of Evolution 02
  - Theory of Evolution 03
  - Theory of Evolution 04
  - Week 3 Feedback Form : The Joy of Computing using Python!!
  - **Quiz: Week 3 : Assignment 3**
  - Week 3 Programming Assignment - 1
  - Week 3 Programming Assignment - 2
  - Week 3 Programming Assignment - 3
  - Week 3 solution
- **week 4**
- **Week 5**
- **Week 6**
- **Week 7**
- **Week 8**
- **Week 9**
- **Week 10**
- **Week 11**
- **Week 12**
- **Text Transcripts**
- **Download Videos**
- **Problem Solving Session - July 2025**

Week 3 : Assignment 3

The due date for submitting this assignment has passed.

Due on 2025-08-13, 23:59 IST.

Assignment submitted on 2025-08-12, 18:08 IST

Satish, the class monitor, maintains a list of students who complete their homework daily. Each day, he adds or removes names from the list depending on who submitted their work. He also checks who was first or last, and sometimes updates a name if he made a spelling mistake.

1) What is the correct way for Satish to start an empty list to record names? *1 point*

○ list = ()
○ list = {}
● list = []
○ list = <>

Yes, the answer is correct.
Score: 1
Accepted Answers:
list = []

2) Satish wants to add "Anya" to his homework list. Which method should he use? *1 point*

○ list.add("Anya")
○ list.push("Anya")
○ list.insert("Anya")
● list.append("Anya")

Yes, the answer is correct.
Score: 1
Accepted Answers:
list.append("Anya")

3) If Satish's list is ["Ravi", "Tina", "Aman"], what will list[0] return? *1 point*

○ "Tina"
○ "Aman"
● "Ravi"
○ 0

Yes, the answer is correct.
Score: 1
Accepted Answers:
"Ravi"

4) To remove "Tina" from the list, Satish should use: *1 point*

○ list.delete("Tina")
● list.remove("Tina")
○ list.pop("Tina")
○ list.cut("Tina")

Yes, the answer is correct.
Score: 1
Accepted Answers:
list.remove("Tina")

5) Satish wants to correct a spelling mistake and replace "Ravi" with "Ravindra" at position 0 in the list students. Which of the following code snippets will accomplish this? *1 point*

☑ students[0] = "Ravindra"
☐ students[0:1] = ["Ravindra"]
☐ students.insert(0, "Ravindra")
☐ students.replace("Ravi", "Ravindra")

Partially Correct.
Score: 0.5
Accepted Answers:
students[0] = "Ravindra"
students[0:1] = ["Ravindra"]

6) If list1 = ["Ravi", "Anya"] and list2 = ["Tina"], what does list1 + list2 return? *1 point*

○ ["RaviAnyaTina"]
● ["Ravi", "Anya", "Tina"]
○ ["Tina", "Ravi", "Anya"]
○ Error

Yes, the answer is correct.
Score: 1
Accepted Answers:
["Ravi", "Anya", "Tina"]

7) What will be the result of ["Aman"] * 3? *1 point*

○ ["Aman3"]
● ["Aman", "Aman", "Aman"]
○ ["Aman3", "Aman3", "Aman3"]
○ Error

Yes, the answer is correct.
Score: 1
Accepted Answers:
["Aman", "Aman", "Aman"]

8) If students = ["Ravi", "Anya", "Tina", "Aman", "Neha"], what does students[1:4] return? *1 point*

○ ["Ravi", "Anya", "Tina", "Aman"]
● ["Anya", "Tina", "Aman"]
○ ["Tina", "Aman", "Neha"]
○ ["Anya", "Tina", "Aman", "Neha"]

Yes, the answer is correct.
Score: 1
Accepted Answers:
["Anya", "Tina", "Aman"]

9) Given students = ["Ravi", "Anya", "Tina", "Aman", "Neha"], what does students[-2:] return? *1 point*

○ ["Tina", "Aman"]
● ["Aman", "Neha"]
○ ["Neha"]
○ Error

Yes, the answer is correct.
Score: 1
Accepted Answers:
["Aman", "Neha"]

10) If students = ["Ravi", "Anya", "Tina", "Aman", "Neha", "Kabir"], what does students[::2] return? *1 point*

● ["Ravi", "Tina", "Neha"]
○ ["Anya", "Aman", "Kabir"]
○ ["Ravi", "Anya", "Kabir"]
○ ["Ravi", "Tina", "Aman", "Kabir"]

Yes, the answer is correct.
Score: 1
Accepted Answers:
["Ravi", "Tina", "Neha"]

Satish is building a backup system to store student homework submissions. Each class's submissions are stored in a list. To avoid losing data, he writes code to create backups of these lists.

At first, Satish copies the list by simply writing:

```
backup = submissions
```

But when he changes the original list, he notices that the backup also changes. Confused, he tries another method:

```
backup = submissions[:]
```

Sometimes it works. But when the list has other lists inside (like group submissions), even that doesn't behave as he expects.

Now Satish is unsure — is he really creating a new list each time?

11) Satish creates a backup like this: *1 point*

```
submissions = ["Ravi", "Tina"]
backup = submissions
submissions.append("Aman")
```

What could explain why both submissions and backup seem to have 3 items now?

☑ Satish created a reference, not a real copy.
☐ The backup was overwritten by append().
☐ backup and submissions point to the same list in memory.
☐ The append() affected only submissions, not backup.

Partially Correct.
Score: 0.5
Accepted Answers:
Satish created a reference, not a real copy.
backup and submissions point to the same list in memory.

12) *1 point*

```
submissions = ["Ravi", "Tina", "Aman"]
backup = submissions[:]
submissions[0] = "Neha"
```

How does this change affect the original and backup lists?

☐ backup[0] still holds "Ravi".
☑ Slicing created a new list with separate memory.
☐ Changing submissions[0] also updates backup[0].
☐ Both lists still refer to the same object.

Partially Correct.
Score: 0.5
Accepted Answers:
backup[0] still holds "Ravi".
Slicing created a new list with separate memory.

13) *1 point*

```
submissions = [["Ravi", "PDF"], ["Tina", "DOC"]]
backup = submissions
submissions[0][1] = "ZIP"
```

What might Satish notice wrt the above code ?

☑ The backup also shows "ZIP" for Ravi.
☐ Changing the submissions list also changes the backup.
☐ The backup list changed because backup and submissions both point to the same list in memory.
☐ The change only applies to submissions, not backup.

Partially Correct.
Score: 0.33
Accepted Answers:
The backup also shows "ZIP" for Ravi.
Changing the submissions list also changes the backup.
The backup list changed because backup and submissions both point to the same list in memory.

14) Satish tries slicing again with nested lists: *1 point*

```
submissions = [["Ravi", "PDF"], ["Tina", "DOC"]]
backup = submissions[:]
submissions[1][1] = "ZIP"
```

What unexpected behavior might confuse Satish?

☐ The backup also shows "ZIP" instead of "DOC".
☑ Slicing didn't protect inner lists from being shared.
☐ The backup list is completely unrelated to the original.
☐ Only the outer list was copied, not the inner lists.

Partially Correct.
Score: 0.33
Accepted Answers:
The backup also shows "ZIP" instead of "DOC".
Slicing didn't protect inner lists from being shared.
Only the outer list was copied, not the inner lists.

15) Satish makes a backup and modifies it: *1 point*

```
submissions = ["Ravi", "Tina"]
backup = submissions[:]
backup.append("Neha")
```

What is the final state of both lists?

☐ submissions has 2 items.
☐ backup has 3 items.
☑ "Neha" appears in both lists.
☐ The original list was also updated with "Neha".

No, the answer is incorrect.
Score: 0
Accepted Answers:
submissions has 2 items.
backup has 3 items.

16) Satish creates two sublists using the same variable: *1 point*

```
inner = ["Ravi"]
submissions = [inner, inner]
submissions[0].append("PDF")
```

What will submissions look like?

○ [["Ravi", "PDF"], ["Ravi"]]
● [["Ravi", "PDF"], ["Ravi", "PDF"]]
○ [["PDF"], ["PDF"]]
○ [["Ravi"], ["Ravi", "PDF"]]

Yes, the answer is correct.
Score: 1
Accepted Answers:
[["Ravi", "PDF"], ["Ravi", "PDF"]]

17) Satish updates the backup list thinking it's separate from the original: *1 point*

```
submissions = ["Ravi"]
backup = submissions
backup[0] = "Neha"
```

What will be the value of submissions[0]?

○ "Ravi"
○ ["Ravi"]
● "Neha"
○ It raises an error

Yes, the answer is correct.
Score: 1
Accepted Answers:
"Neha"

18) Satish tries this slice: *1 point*

```
submissions = ["Ravi", "Tina", "Aman"]
backup = submissions[2:1]
```

What is the value of backup?

● ["Aman", "Tina"]
○ ["Tina"]
○ []
○ It raises an error

No, the answer is incorrect.
Score: 0
Accepted Answers:
[]

19) Satish modifies the backup reference like this: *1 point*

```
submissions = ["Ravi", "Tina"]
backup = submissions
backup = ["Aman"]
```

What is the final value of submissions?

○ ["Aman"]
○ ["Ravi", "Tina"]
● []
○ ["Ravi"]

No, the answer is incorrect.
Score: 0
Accepted Answers:
["Ravi", "Tina"]

20) Satish slices a list of lists and modifies the first sublist: *1 point*

```
group1 = [["Ravi"], ["Tina"]]
group2 = group1[:]
group1[0].append("Submitted")
```

What is the value of group2[0]?

○ ["Ravi"]
○ ["Submitted"]
● ["Ravi", "Submitted"]
○ It raises an error

Yes, the answer is correct.
Score: 1
Accepted Answers:
["Ravi", "Submitted"]

At IIT Ropar, students loved their lunchtime. But with hundreds of students and dozens of meal combinations, chaos ruled the canteen. To solve this, the institute launched **Smart Canteen 2.0**, a digital system built entirely using Python ,with the help of five student programmers.

Each student took on a part of the system:

Nishanur: The Pairing Expert (zip())

Nishanur had to match student names with the meals they ordered. He had two lists , one of names and one of meals. At first, he used a loop with indexing to pair them, but it was messy.

Then, he discovered:

```
zip(names, meals)
```

This zipped them together into clean pairs like ("Ravi", "Idli"), ("Anya", "Poha").

Maria : The Counter (enumerate())

Maria wanted to add serial numbers while displaying the orders. She was looping with range(len(list)), until she found:

```
enumerate(names)
```

This gave her both the index and the item in a cleaner way — like (0, 'Ravi'), (1, 'Anya').

Saujanya: **The Calculator (map())**

Saujanya had to add **18% GST** to every meal price. Instead of a loop with manual math, she used:

```
map(lambda price: price * 1.18, prices)
```

This means the same calculation (adding 18% GST) is applied to each price in the list, one by one, automatically, without writing a loop. The result is a new list with all the updated prices.

Aashik: The Filter Master (filter())

Aashik needed to show only vegetarian meals. He created a list with all meals and used:

```
filter(lambda meal: meal in veg_items, all_meals)
```

This picked only the meals that matched his condition.

map() applies a function to all items in a list.

What is lambda?

A lambda is a quick, one-line function used without def. For example:

```
lambda x: x * 2
```

which is equivalent to :

```
def double(x): return x * 2
```

Their mentor, the professor, smiled and said:
"You all just used iterator functions — tools that let you loop over data more elegantly. These are the LEGO blocks of Python!"

21) Suppose names = ["Ravi", "Anya"] and meals = ["Idli", "Poha"]. What will the following expression do? *1 point*

```
zip(names, meals)
```

☑ It pairs each name with a corresponding meal
☐ It changes the original names and meals lists
☐ It creates a sequence of grouped elements
☐ It returns a list of values from the second list only

Partially Correct.
Score: 0.5
Accepted Answers:
It pairs each name with a corresponding meal
It creates a sequence of grouped elements

22) Which of the following advantages of enumerate() are clearly mentioned or shown in the comprehension? *1 point*

☑ Automatically gives both index and value
☐ Helps avoid using range(len(...))
☐ Returns reversed values
☐ Removes duplicate items from the list

Partially Correct.
Score: 0.5
Accepted Answers:
Automatically gives both index and value
Helps avoid using range(len(...))

23) Which of the following best describes a scenario where using map() would be preferred over a for loop? *1 point*

○ When you want to pair student names with their meals
○ When you want to display items along with their serial numbers
● When you want to apply a calculation to every item in a list without modifying the original
○ When you want to remove all non-vegetarian meals from a list

Yes, the answer is correct.
Score: 1
Accepted Answers:
When you want to apply a calculation to every item in a list without modifying the original

24) What will be the output of the following code? *1 point*

```
names = ['Ravi', 'Anya', 'Sita']
meals = ['Idli', 'Poha', 'Dosa']
pairs = zip(names, meals)
names[0] = 'Amit'
print(list(pairs))
```

● [('Ravi', 'Idli'), ('Anya', 'Poha'), ('Sita', 'Dosa')]
○ [('Amit', 'Idli'), ('Anya', 'Poha'), ('Sita', 'Dosa')]
○ [('Amit', 'Poha'), ('Anya', 'Dosa')]
○ [('Amit', 'Poha'), ('Sita', 'Dosa')]

No, the answer is incorrect.
Score: 0
Accepted Answers:
[('Amit', 'Idli'), ('Anya', 'Poha'), ('Sita', 'Dosa')]

25) What is the output of this code? *1 point*

```
data = ['Idli', 'Poha', 'Dosa']
for i, item in enumerate(data):
    data[i] = item.upper()
print(data)
```

● ['Idli', 'Poha', 'Dosa']
○ ['IDLI', 'POHA', 'DOSA']
○ ['IDLI', 'Poha', 'Dosa']
○ ['Idli', 'POHA', 'DOSA']

No, the answer is incorrect.
Score: 0
Accepted Answers:
['IDLI', 'POHA', 'DOSA']

26) Predict the output: *1 point*

```
prices = [100, 200]
updated_prices = list(map(lambda p: p * 1.18, prices))
prices[1] = 300
print(updated_prices)
```

● [118.0, 236.0]
○ [118.0, 354.0]
○ [100, 300]
○ [118, 300]

Yes, the answer is correct.
Score: 1
Accepted Answers:
[118.0, 236.0]

27) What will the following code return? *1 point*

```
meals = ['Idli', 'Poha', 'Chicken']
veg_items = ['Idli', 'Poha', 'Paneer']
result = filter(lambda x: x not in veg_items, meals)
print(len(list(result)))
```

● 0
○ 1
○ 2
○ 3

No, the answer is incorrect.
Score: 0
Accepted Answers:
1

28) What will be the final content of new_prices? *1 point*

```
meal_prices = [100, 150, 200]
gst_func = lambda p: p * 1.18
new_prices = list(map(gst_func, meal_prices[:2]))
meal_prices[1] = 300
print(new_prices)
```

○ [118.0, 177.0]
○ [118.0, 354.0]
● [118.0, 177.0, 236.0]
○ [118.0, 300.0]

No, the answer is incorrect.
Score: 0
Accepted Answers:
[118.0, 177.0]

29) What will be printed? *1 point*

```
data = [10, 20, 30]
for i, val in enumerate(data):
    val = val + 5
print(data)
```

○ [15, 25, 35]
● [10, 20, 30]
○ [10, 25, 30]
○ Error

Yes, the answer is correct.
Score: 1
Accepted Answers:
[10, 20, 30]

30) Choose the correct output: *1 point*

```
items = ['a', 'b', 'c']
result = list(map(lambda x: x + '1', items))
print(''.join(result))
```

○ abc
● ['a1', 'b1', 'c1']
○ a1b1c1
○ Error

No, the answer is incorrect.
Score: 0
Accepted Answers:
a1b1c1